

Parte 1 — Paradigmas e conceitos básicos de POO

Questão 1. (assinalar)

Sobre os conceitos fundamentais de classe e objeto, analise as afirmações:

- I. Uma classe funciona como um modelo (molde) a partir do qual objetos são criados.
- II. Um objeto é uma instância de uma classe e possui estado próprio.
- III. O estado de um objeto é definido pelos valores de seus atributos em um dado momento.
- IV. Todos os objetos de uma mesma classe compartilham exatamente o mesmo estado.

Assinale a alternativa correta:

- ☒ a) Apenas I, II e III estão corretas.
- ☐ b) Apenas I e IV estão corretas.
- ☐ c) Apenas II e IV estão corretas.
- ☐ d) Todas as afirmações estão corretas.
- ☐ e) Apenas III está correta.

Questão 2. (somatória)

Sobre os objetivos do paradigma orientado a objetos e suas características, some os números das afirmativas CORRETAS:

- ☒ 01. Um dos objetivos é melhorar a compreensão do sistema, facilitando manutenção e evolução.
- ☒ 02. Busca um alto grau de reutilização por meio de componentes acopláveis.
- ☐ 04. Garante que todo programa OO terá menos linhas que a versão imperativa equivalente.
- ☒ 08. Organiza o software em objetos que combinam dados e comportamentos.
- ☐ 16. Dispensa completamente qualquer etapa de modelagem antes de programar.

Resposta (soma): 11 (01 + 02 + 08)

Questão 3. (assinalar)

Um paradigma de programação determina a forma como o programador estrutura e executa o programa.

Assinale a alternativa que associa corretamente paradigma e característica:

- ☐ a) Funcional: baseia-se em comandos sequenciais que alteram o estado.
- ☐ b) Lógico: trata a computação como avaliação de funções matemáticas.
- ☒ c) Orientado a objetos: organiza o código em objetos, instâncias de classes.
- ☐ d) Imperativo: baseia-se em fatos e regras, como o Prolog.
- ☐ e) Orientado a objetos: não pode ser utilizado na linguagem Java.

Parte 2 — UML e diagrama de classes

Questão 4. (somatória)

Sobre a UML, o diagrama de classes e as convenções de nomenclatura, some os números das afirmativas CORRETAS:

- ☒ 01. A UML é uma linguagem visual para modelar sistemas orientados a objetos.
- ☒ 02. Nomes de classe usam UpperCamelCase e representam substantivos do vocabulário do sistema.
- ☒ 04. Nomes de atributo usam lowerCamelCase.
- ☐ 08. Nomes de operação são substantivos, nunca verbos.



() 16. É obrigatório utilizar todos os 14 diagramas da UML em qualquer projeto.

Resposta (soma): 7 (01 + 02 + 04)

Questão 5. (relacionar colunas)

Relacione cada tipo de dado Java à sua descrição e assinale a sequência lida de cima para baixo na coluna da direita:

- | | |
|------------|--|
| A. boolean | (C) Representa texto. |
| B. double | (A) Tipo lógico (verdadeiro ou falso). |
| C. String | (B) Representa números decimais. |

Resposta (sequência de cima para baixo): C - A - B

Questão 6. (assinalar)

No diagrama de classes, uma classe é dividida em compartimentos. Assinale a alternativa correta sobre esses elementos:

- () a) O primeiro compartimento contém as operações da classe.
- (X) b) Os atributos representam propriedades da classe e recebem nomes em lowerCamelCase.
- () c) As operações não podem ter parênteses após o nome.
- () d) O nome da classe deve ser escrito em lowerCamelCase.
- () e) Atributos e operações são sinônimos em UML.

Parte 3 — Implementação, memória e diagrama de objetos

Questão 7. (somatória)

Sobre variáveis de referência, o operador new e as áreas de memória, some os números das afirmativas CORRETAS:

- (X) 01. Uma variável de referência armazena o endereço de um objeto, não o objeto em si.
- (X) 02. Os objetos são armazenados na área de memória Heap.
- (X) 04. A área Stack guarda variáveis locais, parâmetros e chamadas de métodos.
- (X) 08. O operador new cria o objeto, inicializa suas variáveis de instância e retorna seu endereço.

- () 16. O operador new é utilizado para criar variáveis do tipo int e double.

Resposta (soma): 15 (01 + 02 + 04 + 08)

Questão 8. (assinalar)

Considere que a classe Produto tem os atributos públicos preco e quantidade e o método calcularTotal(). Analise:

```
Produto p = new Produto();  
p.preco = 10.0;  
p.quantidade = 3;  
double total = p.calcularTotal();
```

Assinale a alternativa correta:

- (X) a) O operador ponto (.) é usado para acessar atributos e invocar métodos do objeto.
- () b) A palavra new é dispensável para criar o objeto p.
- () c) A variável p guarda diretamente os valores dos atributos, e não um endereço.
- () d) A linha "double total = p.calcularTotal();" causa erro de compilação.
- () e) O método calcularTotal() exige preco e quantidade como parâmetros para funcionar.

Questão 9. *(somatória)*

Sobre a tradução de um diagrama de classes para código Java, some os números das afirmativas CORRETAS:

- ☒ 01. Atributos costumam se tornar variáveis de instância.
- ☒ 02. Operações costumam se tornar métodos.
- ☐ 04. Métodos são proibidos de utilizar os atributos do próprio objeto.
- ☒ 08. Um método que apenas altera o estado, sem devolver valor, é declarado como void.
- ☒ 16. Um método que devolve um valor deve declarar o tipo do dado retornado.

Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 10. *(assinalar)*

Comparando o diagrama de classes e o diagrama de objetos, assinale a alternativa correta:

- ☐ a) O diagrama de objetos mostra a estrutura estática das classes e seus relacionamentos.
- ☐ b) O diagrama de classes é uma fotografia do estado do sistema em um instante.
- ☒ c) O diagrama de objetos representa uma fotografia do sistema em um instante, com objetos e seus valores.
- ☐ d) Ambos representam exatamente a mesma informação.
- ☐ e) O diagrama de objetos não pode conter valores de atributos.

Parte 4 — Escopo de variáveis

Questão 11. *(assinalar)*

Considere:

```
public class Contador {  
    int total = 50;  
    void somar() {  
        int total = 5;  
        total = total + 1;  
        System.out.println(total);  
        System.out.println(this.total);  
    }  
}
```

Ao chamar somar(), o que é impresso?

- ☒ a) 6 e depois 50.
- ☐ b) 51 e depois 6.
- ☐ c) 6 e depois 6.
- ☐ d) 50 e depois 50.
- ☐ e) Erro de compilação, pois há duas variáveis total.

Questão 12. *(somatória)*

Sobre escopo, inicialização e valores padrão de variáveis, some os números das afirmativas CORRETAS:

- ☒ 01. Uma variável de instância tem como escopo todos os métodos da classe.
- ☒ 02. Uma variável local tem como escopo apenas o método em que foi declarada.
- ☐ 04. Uma variável local pode ser lida sem ter recebido uma atribuição explícita.
- ☒ 08. Uma variável de instância numérica é inicializada automaticamente com 0.
- ☒ 16. Uma variável de instância de referência é inicializada automaticamente com null.



Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 13. (assinalar)

Considere:

```
public class Exemplo {  
    void executar() {  
        int i = 0;  
        while (i < 3) {  
            int soma = i * 2;  
            i++;  
        }  
        System.out.println(soma);  
    }  
}
```

Assinale a alternativa correta:

- () a) O código compila e imprime 4.
- () b) O código compila e imprime 0.
- (X) c) Erro de compilação: soma existe apenas dentro do bloco do while e está fora de escopo no println.**
- () d) O código imprime 6.
- () e) Erro de compilação porque i não pode ser incrementado.

Parte 5 — Encapsulamento

Questão 14. (somatória)

Sobre encapsulamento e métodos de acesso (getters e setters), some os números das afirmativas CORRETAS:

- (X) 01. Encapsular um atributo significa, tipicamente, torná-lo private e controlar o acesso por métodos.**
- (X) 02. Um getter recupera o valor de um atributo, não tem parâmetro e retorna um valor.**
- () 04. Deixar todos os atributos públicos é a forma recomendada de encapsulá-los.
- (X) 08. Um setter atribui valor a um atributo, possui um parâmetro e normalmente é void.**
- (X) 16. Para um atributo booleano, o getter pode usar o prefixo "is".**

Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 15. (assinalar)

Uma classe ContaCorrente possui o atributo saldo com acesso público, permitindo que outra classe faça conta.saldo = 999999; sem qualquer depósito. Assinale a alternativa que descreve o problema e a solução corretos:

- () a) Não há problema; atributos públicos preservam a integridade dos dados.
- (X) b) O problema é o acesso direto; a solução é tornar saldo privado e controlar o acesso por métodos.**
- () c) A solução correta é declarar o atributo saldo como static.
- () d) O problema desaparece apenas com a criação de um construtor.
- () e) A solução é remover o atributo saldo da classe.

Questão 16. (assinalar)

Considere o método de acesso:

```
private String nome;  
public void setNome(String nome) {  
    this.nome = nome;  
}
```



```
}
```

Qual o papel da palavra `this` nesse trecho?

- ☐ a) Torna o atributo `nome` uma variável de classe (`static`).
- ☐ b) Refere-se ao parâmetro recebido pelo método.
- ☐ c) É opcional e não faz diferença alguma nesse caso.
- ☒ d) Refere-se à variável de instância `nome`, distinguindo-a do parâmetro de mesmo nome.
- ☐ e) Provoca erro de compilação por conflito de nomes.

Questão 17. *(relacionar colunas)*

Relacione o símbolo de visibilidade da UML ao seu significado e assinale a sequência da coluna da direita (de cima para baixo):

- | | |
|------------------|--|
| A. + (mais) | ☒ B) <code>private</code> — somente a própria classe. |
| B. - (menos) | ☒ A) <code>public</code> — qualquer classe. |
| C. # (sustenido) | ☒ C) <code>protected</code> — protegido. |

Resposta (sequência de cima para baixo): B - A - C

Parte 6 — Membros de classe, sobrecarga e construtores

Questão 18. *(soma)*

Sobre construtores em Java, some os números das afirmativas CORRETAS:

- ☒ 01. O construtor é invocado durante a criação do objeto e serve para inicializá-lo.
- ☒ 02. O construtor tem o mesmo identificador da classe e não declara tipo de retorno.
- ☒ 04. Se nenhum construtor for implementado, o compilador fornece um construtor padrão sem argumentos.
- ☐ 08. Todo construtor deve obrigatoriamente declarar `void` como retorno.
- ☒ 16. O operador `new` executa o construtor ao criar o objeto.

Resposta (soma): 23 (01 + 02 + 04 + 16)

Questão 19. *(assinalar)*

Considere os métodos na mesma classe:

```
double media(int a, int b) { ... }  
double media(double a, double b) { ... }
```

Assinale a alternativa correta sobre sobrecarga:

- ☒ a) É uma sobrecarga válida, pois as listas de parâmetros (assinaturas) são diferentes.
- ☐ b) Não é válida, pois os nomes dos métodos são iguais.
- ☐ c) Não é válida, pois o tipo de retorno é o mesmo.
- ☐ d) Só é válida se um dos métodos for declarado como `static`.
- ☐ e) Não é válida, pois sobrecarga não permite tipos `double`.

Questão 20. *(soma)*

Sobre membros de classe (`static`) e sobrecarga, some os números das afirmativas CORRETAS:

- ☒ 01. Uma variável `static` é compartilhada por todas as instâncias da classe.
- ☒ 02. Um método `static` não pode utilizar a palavra `this`.
- ☒ 04. Sobrecarga é ter métodos de mesmo nome, na mesma classe, com assinaturas diferentes.

- () 08. O compilador diferencia métodos sobrecarregados apenas pelo tipo de retorno.
() 16. Uma variável static é exclusiva de cada instância, nunca compartilhada.

Resposta (soma): **7** (01 + 02 + 04)

Parte 7 — Lançamento e tratamento de exceções

Questão 21. (assinalar)

Considere:

```
public void sacar(double valor) {  
    if (valor > saldo) {  
        throw new IllegalArgumentException("Saldo insuficiente");  
    }  
    saldo = saldo - valor;  
}
```

Assinale a alternativa correta:

- (X) a) Se valor for maior que saldo, uma exceção é lançada e a subtração não é executada.**
() b) A palavra throw serve para capturar (tratar) a exceção.
() c) O método sempre lança a exceção, independentemente do valor.
() d) A mensagem "Saldo insuficiente" não pode ser recuperada no tratamento.
() e) O código não compila, pois throw não existe em Java.

Questão 22. (somatória)

Sobre try..catch..finally, hierarquia e propagação de exceções, some os números das afirmativas CORRETAS:

- (X) 01. Um erro em um comando do bloco try desvia o fluxo para um bloco catch compatível.**
() 02. A cláusula finally só é executada quando nenhuma exceção ocorre.
(X) 04. Se um método não trata a exceção, ela é propagada ao método chamador.
(X) 08. A classe Error indica erros graves, que normalmente não se espera tratar no programa.
() 16. A cláusula finally nunca é executada quando há um bloco catch.

Resposta (soma): **13** (01 + 04 + 08)

Questão 23. (assinalar)

Considere:

```
try {  
    String s = null;  
    System.out.println(s.length());  
} catch (NullPointerException e) {  
    System.out.println("X");  
} finally {  
    System.out.println("Y");  
}
```

Qual é a saída?

- () a) Apenas X
() b) Apenas Y
(X) c) X e depois Y
() d) Nenhuma saída; o programa encerra com erro.
() e) Erro de compilação.



Questão 24. *(relacionar colunas)*

Relacione cada mecanismo de exceção à sua descrição e assinale a sequência da coluna da direita (de cima para baixo):

- | | |
|---------------------------------------|---|
| A. try | (C) Cláusula que delega ao método chamador o tratamento da exceção. |
| B. catch | (A) Bloco que contém os comandos que podem lançar uma exceção. |
| C. throws | (D) Subclasse de RuntimeException; não obriga tratamento explícito. |
| D. Exceção não verificada (unchecked) | (B) Bloco que captura e trata a exceção lançada. |

Resposta (sequência de cima para baixo): C - A - D - B

Parte 8 — Qualidade de software e testes de unidade

Questão 25. *(soma)*

Sobre qualidade de software, testes de unidade e JUnit, some os números das afirmativas CORRETAS:

- (X) 01. Detectar erros mais cedo no ciclo de desenvolvimento reduz o custo de correção.
- (X) 02. Em POO, a unidade avaliada em um teste de unidade é o método.
- () 04. assertThrows serve para testar a performance de um método em produção.
- (X) 08. No JUnit, o método de teste recebe @Test, é público, sem parâmetros e retorna void.
- (X) 16. assertEquals compara um valor esperado com o valor obtido.

Resposta (soma): 27 (01 + 02 + 08 + 16)